

#5 - Directory Favorites & Full Filters

What & Why

The directory page needs a real favorites system (heart button that persists to the database), a dedicated Favorites section at the top of the results, a Favorites-only filter toggle, a text search field, and the full expanded lists of professions and genres that were provided previously.

Done looks like

- A text search input appears in the filter bar so users can search by name, skill, or keyword
- The Profession and Genre dropdowns contain the complete lists of all options (not the trimmed 6-item versions)
- Every user card has a heart button — when pressed while logged in, it persists to the database (POST `/api/favorites` or DELETE `/api/favorites/:userId/:favoriteUserId`) and the heart fills/unfills immediately
- A "Favorites" filter toggle button appears in the filter bar; clicking it shows ONLY favorited users
- When a user has any favorites, a dedicated "Your Favorites" section appears above the main results list, showing those favorited users' cards
- If not logged in, the heart button still shows but tapping it prompts nothing (graceful degradation — no crash)

Out of scope

- Changing the backend or schema (all necessary API routes already exist)
- Changing any other page besides `directory-new.tsx`

Tasks

1. **Wire heart button to the API** — Read `currentUserId` from `localStorage`; on mount fetch `GET /api/favorites/:userId` to initialize the favorited IDs set; add/remove mutations using `POST /api/favorites` and `DELETE /api/favorites/:userId/:favoriteUserId`; invalidate the favorites query after each mutation.
2. **Add search input & expand filter lists** — Add a text search `<Input>` to the filter bar; replace the truncated 6-item profession and genre lists with the full lists from

`directory.tsx`; apply the search term, city, profession, and genre filters together in `filterUsers()`.

3. **Add Favorites filter toggle & Favorites section** — Show the hidden "Favorites" button in the filter bar; when active, `filteredUsers` is restricted to favorited IDs only; render a "Your Favorites" section above the main list showing only favorited users when the user is logged in and has at least one favorite.

Relevant files

- `client/src/pages/directory-new.tsx`
- `client/src/pages/directory.tsx:27-69`
- `server/routes.ts:324-368`

```
import { useState, useMemo } from "react";
import { Link } from "wouter";
import { useQuery, useMutation } from "@tanstack/react-query";
import { queryClient, apiRequest } from "@lib/queryClient";
import { Card, CardContent } from "@components/ui/card";
import { Button } from "@components/ui/button";
import { Input } from "@components/ui/input";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from
"@components/ui/select";
import { Heart, MapPin, Users, Music2, Loader2, Search } from "lucide-react";
import { Badge } from "@components/ui/badge";
import { getGenreBadge, getProfessionBadge } from "@lib/badges";
import goldBadge from "@assets/Gold_Level-removebg-preview_1762468528106.png";
import platinumBadge from "@assets/Platinum Level_1762468203581.png";
import type { User, Favorite } from "@shared/schema";
```

```
const getUserSlug = (user: User) =>
  `${user.firstName}-${user.lastName}`.toLowerCase().replace(/\s+/g, "-");
```

```
export default function DirectoryPage() {
  const [searchQuery, setSearchQuery] = useState("");
  const [city, setCity] = useState("all");
  const [profession, setProfession] = useState("all");
  const [genre, setGenre] = useState("all");
  const [showFavoritesOnly, setShowFavoritesOnly] = useState(false);
```

```
const currentUserId = localStorage.getItem("currentUserId")
  ? parseInt(localStorage.getItem("currentUserId")!)
  : null;
```

```
const { data: users = [], isLoading } = useQuery<User[]>({
```

```
    queryKey: ["/api/users"],
  });
```

```
const { data: favoritesData = [] } = useQuery<Favorite[]>({
  queryKey: ["/api/favorites", currentUserId],
  queryFn: async () => {
    if (!currentUserId) return [];
    const res = await fetch(`/api/favorites/${currentUserId}`);
    if (!res.ok) return [];
    return res.json();
  },
  enabled: !!currentUserId,
});
```

```
const favoritedIds = useMemo(
  () => new Set(favoritesData.map((f) => f.favoriteUserId)),
  [favoritesData]
);
```

```
const favQKey = ["/api/favorites", currentUserId];
```

```
const addFavoriteMutation = useMutation({
  mutationFn: (favoriteUserId: number) =>
    apiRequest("POST", "/api/favorites", { userId: currentUserId, favoriteUserId }),
  onMutate: async (favoriteUserId: number) => {
    await queryClient.cancelQueries({ queryKey: favQKey });
    const previous = queryClient.getQueryData<Favorite[]>(favQKey) ?? [];
    const optimistic: Favorite = { id: -1, userId: currentUserId!, favoriteUserId, createdAt: new
Date() };
    queryClient.setQueryData<Favorite[]>(favQKey, [...previous, optimistic]);
    return { previous };
  },
  onError: (_err, _vars, ctx) => {
    if (ctx?.previous) queryClient.setQueryData(favQKey, ctx.previous);
  },
  onSettled: () => {
    queryClient.invalidateQueries({ queryKey: favQKey });
  },
});
```

```
const removeFavoriteMutation = useMutation({
  mutationFn: (favoriteUserId: number) =>
    apiRequest("DELETE", `/api/favorites/${currentUserId}/${favoriteUserId}`),
  onMutate: async (favoriteUserId: number) => {
```

```

    await queryClient.cancelQueries({ queryKey: favQKey });
    const previous = queryClient.getQueryData<Favorite[]>(favQKey) ?? [];
    queryClient.setQueryData<Favorite[]>(favQKey, previous.filter((f) => f.favoriteUserId !==
favoriteUserId));
    return { previous };
  },
  onError: (_err, _vars, ctx) => {
    if (ctx?.previous) queryClient.setQueryData(favQKey, ctx.previous);
  },
  onSettled: () => {
    queryClient.invalidateQueries({ queryKey: favQKey });
  },
});

```

```

const pendingFavoriteId =
  (addFavoriteMutation.isPending ? addFavoriteMutation.variables : null) ??
  (removeFavoriteMutation.isPending ? removeFavoriteMutation.variables : null);

```

```

const toggleFavorite = (e: React.MouseEvent, userId: number) => {
  e.preventDefault();
  e.stopPropagation();
  if (!currentUserId) return;
  if (pendingFavoriteId === userId) return;
  if (favoritedIds.has(userId)) {
    removeFavoriteMutation.mutate(userId);
  } else {
    addFavoriteMutation.mutate(userId);
  }
};

```

```

const cities = [
  { value: "all", label: "All Cities" },
  { value: "los-angeles", label: "Los Angeles" },
  { value: "new-york", label: "New York" },
  { value: "chicago", label: "Chicago" },
  { value: "miami", label: "Miami" },
  { value: "seattle", label: "Seattle" },
  { value: "sacramento", label: "Sacramento" },
  { value: "nashville", label: "Nashville" },
  { value: "atlanta", label: "Atlanta" },
];

```

```

const professions = [
  { value: "all", label: "All Professions" },

```

```

    { value: "Artist", label: "Artist - Singer, Rapper, Performer" },
    { value: "Administration", label: "Administration - Registrations, Royalties, Licensing,
Copyrights" },
    { value: "Audio", label: "Audio - Engineering, Mixing, Recording, Live Sound, Mastering" },
    { value: "Consultant", label: "Consultant - Coaching, Development, Promotions, Marketing,
Strategy" },
    { value: "Dancer", label: "Dancer - Dancers, Choreographers" },
    { value: "DJ", label: "DJ - Clubs, Parties, Radio" },
    { value: "Educator", label: "Educator - Teachers, Professors, Educators" },
    { value: "Fashion", label: "Fashion - Designers, Stylists" },
    { value: "Glam", label: "Glam - Hair Stylists, Makeup Artists" },
    { value: "Legal", label: "Legal - Lawyers, Attorneys, Legal Affairs, Negotiations" },
    { value: "Management", label: "Management - Artist, Business, Financial, Touring" },
    { value: "Marketing", label: "Marketing - Digital Marketing, Social Media, PR, Branding" },
    { value: "Music Executive", label: "Music Executive - A&R, Label Reps, GMs, Presidents,
VPs" },
    { value: "Musician", label: "Musician - Sessions, Background, Touring" },
    { value: "Photographer/Videographer", label: "Photographer/Videographer - Photos, Videos,
Editing" },
    { value: "Producer", label: "Producer - Beat Maker, Executive Producer" },
    { value: "Publishing", label: "Publishing - Music Publishing" },
    { value: "Radio/Podcast", label: "Radio/Podcast - Host, Producers, Personalities" },
    { value: "Record Label", label: "Record Label - Runs or Works at a Label" },
    { value: "Recording Studio", label: "Recording Studio - Owns or Works at Recording Space" },
    { value: "Songwriter", label: "Songwriter - Writes and Composes Music/Lyrics" },
    { value: "Synch", label: "Synch - Music Supervisors, Synch Reps, Licensors" },
    { value: "Touring", label: "Touring - Agents, Roadies, Stage Managers, Crew" },
    { value: "Venue", label: "Venue - Bars, Clubs, Creative Spaces" },
];

```

```

const genres = [
    { value: "all", label: "All Genres" },
    { value: "Pop", label: "Pop" },
    { value: "Hip-Hop", label: "Hip-Hop / Rap" },
    { value: "R&B", label: "R&B / Soul" },
    { value: "Rock", label: "Rock" },
    { value: "Country", label: "Country / Folk / Americana" },
    { value: "Electronic", label: "Electronic / EDM / Techno" },
    { value: "Dance", label: "Dance / House" },
    { value: "Reggae", label: "Reggae / Dancehall" },
    { value: "Latin", label: "Latin (Reggaetón, Bachata, Salsa)" },
    { value: "Afrobeats", label: "Afrobeats / Afro-Fusion" },
    { value: "Classical", label: "Classical / Opera" },
    { value: "Jazz", label: "Jazz" },
];

```

```

    { value: "Blues", label: "Blues" },
    { value: "Gospel", label: "Gospel / Christian / Inspirational" },
  ];

```

```

const filterUsers = (userList: User[]) => {
  return userList.filter((user) => {
    const q = searchQuery.toLowerCase();
    const nameMatch =
      !q ||
      `${user.firstName} ${user.lastName}`.toLowerCase().includes(q) ||
      (user.pkaName || "").toLowerCase().includes(q) ||
      (user.location || "").toLowerCase().includes(q) ||
      user.profession.some((p) => p.toLowerCase().includes(q)) ||
      user.genre.some((g) => g.toLowerCase().includes(q)) ||
      user.skills.some((s) => s.toLowerCase().includes(q));

    const userLocation = user.location || "";
    const cityMatch =
      city === "all" ||
      userLocation.toLowerCase().replace(/\s+/g, "-").includes(city);

    const professionMatch =
      profession === "all" ||
      user.profession.some((p) => p.toLowerCase() === profession.toLowerCase());

    const genreMatch =
      genre === "all" ||
      user.genre.some((g) => g.toLowerCase().includes(genre.toLowerCase()));

    return nameMatch && cityMatch && professionMatch && genreMatch;
  });
};

```

```

const favoriteUsers = useMemo(
  () => users.filter((u) => favoritedIds.has(u.id)),
  [users, favoritedIds]
);

```

```

const allFiltered = filterUsers(users);
const filteredUsers = showFavoritesOnly
  ? allFiltered.filter((u) => favoritedIds.has(u.id))
  : allFiltered;

```

```

const UserCard = ({ user }: { user: User }) => {

```

```

const displayName =
  user.usePkaAsMain && user.pkaName
  ? user.pkaName
  : `${user.firstName} ${user.lastName}`;
const initials = `${user.firstName.charAt(0)}${user.lastName.charAt(0)}`.toUpperCase();
const primaryProfession = user.profession[0] || "Professional";
const primaryGenre = user.genre[0] || "Various";
const isFav = favoritedIds.has(user.id);

return (
  <Link href={` /profile/${getUserSlug(user)} `}>
    <Card
      data-testid={` card-user-${user.id}`}
      className="bg-gradient-to-br from-white via-purple-50/30 to-white border
border-purple-200/40 hover:border-purple-400 transition-all duration-300 cursor-pointer
rounded-2xl hover:scale-[1.02] transform"
      style={{
        boxShadow: "0 15px 45px -18px rgba(192,132,252,0.25)",
        background:
          "linear-gradient(135deg, color-mix(in oklab, #c084fc 18%, transparent), transparent
55%)",
      }}
    >
      <CardContent className="p-4 relative bg-gradient-to-r from-purple-50 to-purple-100
rounded-2xl">
        <div className="flex items-start justify-between mb-3">
          <div className="flex items-center gap-3">
            <div className="w-12 h-12 bg-gradient-to-br from-purple-500 to-purple-700
rounded-full flex items-center justify-center shadow-lg ring-2 ring-purple-200">
              <span className="text-white font-bold text-sm">{initials}</span>
            </div>
          </div>
          <div className="flex items-center gap-2">
            <h3 className="text-gray-800 font-bold text-lg">{displayName}</h3>
            {user.memberLevel === "Platinum" && (
              <img src={platinumBadge} alt="Platinum Member" className="w-5 h-5
object-contain" />
            )}
            {user.memberLevel === "Gold" && (
              <img src={goldBadge} alt="Gold Member" className="w-5 h-5 object-contain" />
            )}
          </div>
          <p className="text-purple-600 font-semibold text-sm">{primaryProfession}</p>
        </div>
      </CardContent>
    </Card>
  </Link>
)

```

```

</div>

<Button
  variant="ghost"
  size="sm"
  data-testid={`button-favorite-${user.id}`}
  disabled={pendingFavoriteId === user.id}
  className={`w-10 h-10 rounded-full border-2 shadow-lg transition-all duration-300
hover:scale-110 disabled:opacity-60 disabled:cursor-not-allowed ${
    isFav
      ? "bg-gradient-to-br from-yellow-400 to-yellow-500 border-yellow-400 text-white
hover:from-yellow-500 hover:to-yellow-600 shadow-yellow-500/30"
      : "border-purple-300 text-purple-400 hover:border-yellow-400 hover:text-yellow-500
bg-white/70 backdrop-blur-sm hover:bg-yellow-50"
  }}
  onClick={(e) => toggleFavorite(e, user.id)}
>
  {pendingFavoriteId === user.id ? (
    <Loader2 className="w-4 h-4 animate-spin" />
  ) : (
    <Heart className={`w-5 h-5 ${isFav ? "fill-current" : ""}`} />
  )}
</Button>
</div>

{user.skills && user.skills.length > 0 && (
  <div className="flex flex-wrap items-center gap-2 mb-3">
    {user.skills.slice(0, 3).map((skill, index) => (
      <Badge
        key={index}
        className="bg-gradient-to-r from-purple-500 to-purple-600 hover:from-purple-600
hover:to-purple-700 text-white text-xs px-3 py-1 rounded-full shadow-lg shadow-purple-500/30
border border-purple-400/20"
      >
        {skill}
      </Badge>
    ))}
  </div>
)}

<div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-3 mb-4">
  <div className="flex items-center gap-3 bg-gradient-to-r from-purple-50 to-purple-100
rounded-xl p-2">

```



```

    <div className="w-8 h-8 bg-gradient-to-br from-purple-500 to-purple-700
rounded-full flex items-center justify-center shadow-lg">
      <MapPin className="w-4 h-4 text-white" />
    </div>
    <div>
      <p className="text-xs text-purple-600 uppercase tracking-wide
font-semibold">Location</p>
      <p className="text-gray-800 text-sm font-bold">{user.location || "Not
specified"}</p>
    </div>
  </div>

```

```

    <div className="flex items-center gap-3 bg-gradient-to-r from-purple-50 to-purple-100
rounded-xl p-2">
      <div className="w-8 h-8 bg-gradient-to-br from-[#c084fc] to-purple-700 rounded-full
flex items-center justify-center p-1 shadow-lg">
        {getGenreBadge(primaryGenre) ? (
          <img
            src={getGenreBadge(primaryGenre)!}
            alt={` ${primaryGenre} badge`}
            className="w-full h-full object-contain"
          />
        ) : (
          <Music2 className="w-4 h-4 text-white" />
        )}
      </div>
      <div>
        <p className="text-xs text-[#c084fc] uppercase tracking-wide
font-semibold">Genre</p>
        <p className="text-gray-800 text-sm font-bold">{primaryGenre}</p>
      </div>
    </div>

```

```

    <div className="flex items-center gap-3 bg-gradient-to-r from-purple-50 to-purple-100
rounded-xl p-2">
      <div className="w-8 h-8 bg-gradient-to-br from-purple-500 to-purple-700
rounded-full flex items-center justify-center shadow-lg">
        <Users className="w-4 h-4 text-white" />
      </div>
      <div>
        <p className="text-xs text-purple-600 uppercase tracking-wide
font-semibold">Connections</p>
        <p className="text-gray-800 text-sm font-bold">{user.connections || 0}</p>
      </div>
    </div>

```

```

    </div>
  </div>

  {(user.bio || user.recentWork) && (
    <div className="pt-4 mt-4 border-t border-purple-200">
      <div className="bg-gradient-to-r from-purple-50 to-white rounded-xl p-3">
        <p className="text-xs text-purple-600 uppercase tracking-wide mb-2
font-semibold">About</p>
        <p className="text-gray-700 text-sm font-medium line-clamp-2">
          {user.bio || user.recentWork}
        </p>
      </div>
    </div>
  )}
</CardContent>
</Card>
</Link>
);
};

return (
  <div className="min-h-screen bg-white text-black">
    <div className="max-w-4xl mx-auto px-4 py-8">
      {/* Header */}
      <div className="mb-8">
        <div
          className="hero-card"
          style={{
            background:
              "linear-gradient(135deg, color-mix(in oklab, #c084fc 18%, transparent), transparent
55%)",
            border: "1px solid rgba(192,132,252,0.25)",
            borderRadius: "20px",
            padding: "28px",
            boxShadow: "0 15px 45px -18px rgba(192,132,252,0.45)",
          }}
        >
          <div
            className="eyebrow"
            style={{
              display: "inline-flex",
              gap: "8px",
              alignItems: "center",
              color: "#c084fc",

```

```

        fontSize: "14px",
        marginBottom: "6px",
    }}
>
    Music Industry Directory
</div>
<h1
  className="title"
  style={{
    fontWeight: 800,
    letterSpacing: "-0.02em",
    margin: "6px 0 8px",
    fontSize: "2.5rem",
    color: "#1f2937",
  }}
>
  MiC · The Playlist
</h1>
<p
  className="subtitle"
  style={{ color: "rgba(17,17,17,0.6)", maxWidth: "720px", margin: 0 }}
>
  Discover and connect with vetted music industry professionals across roles, genres,
and cities.
</p>
</div>
</div>

```

```

{ /* Search & Filter Card */
  <Card className="bg-white/80 backdrop-blur-sm border border-purple-200 shadow-lg
shadow-purple-500/20 hover:shadow-xl hover:shadow-purple-500/30 rounded-2xl mb-6
transition-all duration-300">
    <CardContent className="p-4 sm:p-6 space-y-4">
      { /* Search input row */
        <div className="relative">
          <Search className="absolute left-3 top-1/2 -translate-y-1/2 text-purple-400 w-4 h-4
pointer-events-none" />
          <Input
            data-testid="input-search"
            placeholder="Search by name, profession, genre, location, or skill..."
            value={searchQuery}
            onChange={(e) => setSearchQuery(e.target.value)}

```

```

        className="pl-10 bg-white border-purple-300 text-gray-700 rounded-full h-10
sm:h-11 text-sm sm:text-base shadow-sm hover:border-purple-400 transition-all
focus-visible:ring-purple-400"
    />
</div>

{/* Dropdown filters row */}
<div className="flex flex-col sm:flex-row gap-3 sm:gap-4">
  <div className="flex-1">
    <Select value={city} onChange={setCity}>
      <SelectTrigger
        data-testid="select-city"
        className="bg-white border-purple-300 text-gray-700 rounded-full h-10 sm:h-11
text-sm shadow-sm hover:border-purple-400 transition-all"
      >
        <SelectValue placeholder="All Cities" />
      </SelectTrigger>
      <SelectContent className="bg-white border-purple-200 shadow-lg">
        {cities.map((c) => (
          <SelectItem key={c.value} value={c.value} className="text-gray-700
hover:bg-purple-50 focus:bg-purple-100">
            {c.label}
          </SelectItem>
        ))}
      </SelectContent>
    </Select>
  </div>

  <div className="flex-1">
    <Select value={profession} onChange={setProfession}>
      <SelectTrigger
        data-testid="select-profession"
        className="bg-white border-purple-300 text-gray-700 rounded-full h-10 sm:h-11
text-sm shadow-sm hover:border-purple-400 transition-all"
      >
        <SelectValue placeholder="All Professions" />
      </SelectTrigger>
      <SelectContent className="bg-white border-purple-200 shadow-lg">
        {professions.map((p) => (
          <SelectItem key={p.value} value={p.value} className="text-gray-700
hover:bg-purple-50 focus:bg-purple-100">
            {p.label}
          </SelectItem>
        ))}
      </SelectContent>
    </Select>
  </div>

```

```

        </SelectContent>
      </Select>
    </div>

    <div className="flex-1">
      <Select value={genre} onChange={setGenre}>
        <SelectTrigger
          data-testid="select-genre"
          className="bg-white border-purple-300 text-gray-700 rounded-full h-10 sm:h-11
text-sm shadow-sm hover:border-purple-400 transition-all"
        >
          <SelectValue placeholder="All Genres" />
        </SelectTrigger>
        <SelectContent className="bg-white border-purple-200 shadow-lg">
          {genres.map((g) => (
            <SelectItem key={g.value} value={g.value} className="text-gray-700
hover:bg-purple-50 focus:bg-purple-100">
              {g.label}
            </SelectItem>
          ))}
        </SelectContent>
      </Select>
    </div>
  </div>

  { /* Filter toggles row */}
  <div className="flex items-center gap-3 flex-wrap">
    <Button
      data-testid="button-filter-favorites"
      variant="ghost"
      size="sm"
      className={`rounded-full px-4 font-medium border transition-all ${
        showFavoritesOnly
          ? "bg-yellow-400 hover:bg-yellow-500 text-white border-yellow-400
shadow-yellow-300/40 shadow-md"
          : "bg-white border-purple-300 text-purple-600 hover:border-yellow-400
hover:text-yellow-600 hover:bg-yellow-50"
      }`}
      onClick={() => setShowFavoritesOnly(!showFavoritesOnly)}
    >
      <Heart className={`w-4 h-4 mr-1.5 ${showFavoritesOnly ? "fill-current" : ""}`} />
      Favorites
      {favoritedIds.size > 0 && (

```

```

        <span className={`ml-1.5 text-xs rounded-full px-1.5 py-0.5 font-bold
${showFavoritesOnly ? "bg-white/30 text-white" : "bg-purple-100 text-purple-700"}`}>
        {favoritedIds.size}
    </span>
    )}
</Button>

```

```

        {(searchQuery || city !== "all" || profession !== "all" || genre !== "all" ||
showFavoritesOnly) && (
    <Button
        data-testid="button-clear-filters"
        variant="ghost"
        size="sm"
        className="rounded-full px-4 text-gray-500 hover:text-gray-700 text-sm"
        onClick={() => {
            setSearchQuery("");
            setCity("all");
            setProfession("all");
            setGenre("all");
            setShowFavoritesOnly(false);
        }}
    >
        Clear filters
    </Button>
)}

```

```

        <span className="ml-auto text-sm text-gray-500">
        <span className="font-semibold text-purple-600">{filteredUsers.length}</span>
members
    </span>
</div>
</CardContent>
</Card>

```

```

{isLoading ? (
    <div className="flex justify-center items-center py-12">
        <Loader2 className="w-8 h-8 animate-spin text-[#c084fc]" />
    </div>
) : (
    <>
        {/* Your Favorites Section */}
        {currentUserId && favoriteUsers.length > 0 && !showFavoritesOnly && (
            <div className="mb-10">
                <div className="flex items-center gap-2 mb-4">

```

```

    <Heart className="w-5 h-5 text-yellow-500 fill-yellow-500" />
    <h2 className="text-lg font-bold text-gray-800">Your Favorites</h2>
    <span className="text-sm text-gray-400">({favoriteUsers.length})</span>
  </div>
  <div className="grid grid-cols-1 gap-4">
    {favoriteUsers.map((user) => (
      <UserCard key={user.id} user={user} />
    ))}
  </div>
  <div className="mt-6 border-t border-purple-100" />
  <div className="mt-6 flex items-center gap-2 mb-4">
    <Users className="w-5 h-5 text-purple-500" />
    <h2 className="text-lg font-bold text-gray-800">All Members</h2>
    <span className="text-sm text-gray-400">({filteredUsers.length})</span>
  </div>
</div>
)}

{/* Main results grid */}
{filteredUsers.length === 0 ? (
  <div className="text-center py-12">
    <Users className="w-16 h-16 text-gray-300 mx-auto mb-4" />
    <h3 className="text-lg font-semibold text-gray-600 mb-2">
      {showFavoritesOnly ? "No favorites yet" : "No professionals found"}
    </h3>
    <p className="text-gray-500">
      {showFavoritesOnly
        ? "Press the heart button on any profile to save them here"
        : "Try adjusting your filters or check back later"}
    </p>
  </div>
) : (
  <div className="grid grid-cols-1 gap-8">
    {filteredUsers.map((user) => (
      <UserCard key={user.id} user={user} />
    ))}
  </div>
)}
</>
)}

{/* Progress Bar */}
{!isLoading && (
  <div className="mt-8">

```

```

    <div className="h-2 bg-purple-200 rounded-full overflow-hidden">
      <div
        className="h-full bg-gradient-to-r from-purple-600 to-purple-500 rounded-full
transition-all duration-300"
        style={{
          width: `${users.length > 0 ? Math.min((filteredUsers.length / users.length) * 100, 100)
: 0}%`,
        }}
      />
    </div>
  </div>
)}

{/* Footer */}
<div className="mt-12 text-center">
  <div className="flex justify-center gap-6 text-purple-400">
    <a href="#" className="hover:underline font-medium">Privacy</a>
    <a href="#" className="hover:underline font-medium">Terms</a>
    <a href="#" className="hover:underline font-medium">Support</a>
  </div>
</div>
</div>
</div>
);
}

```

#6 - Show Heart Button Only When Logged In

What & Why

The favorites heart button currently shows on every directory card for all visitors, including guests who are not logged in. When a guest clicks it, nothing happens silently. This is confusing — guests should see a prompt to log in or the button should be hidden entirely.

Done looks like

- Logged-in users see the heart button and can favorite/unfavorite as expected
- Guests (not logged in) either don't see the button or see a tooltip/redirect prompting them to log in when clicked

Relevant files

- `client/src/pages/directory-new.tsx` (UserCard component, line ~155-175, `toggleFavorite` function)

#7 - Persist Favorites on Profile Pages

What & Why

The heart/favorite button was wired to the API in the directory, but the individual profile page (`profile.tsx`) still has its own separate concept of "following" a user. The favorites system (heart = bookmark) and the follow/connect system are separate, but a logged-in user visiting a profile page doesn't see whether they've favorited that person from the directory.

Done looks like

- A favorite (heart) button appears on individual profile pages
- It reflects the same persisted state as the directory heart button
- Pressing it on a profile page adds/removes the person from the user's favorites list and shows up immediately in the directory's Favorites section